

KVO-LLM: Boosting Long-Context Generation Throughput for Batched LLM Inference

Zhenyu Li[†], Dongxu Lyu[†], Gang Wang, Yuzhou Chen, Liyan Chen, Wenjie Li, Jianfei Jiang, Yanan Sun, Guanghui He*

State Key Laboratory of Micro/Nano Engineering Science, School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China

{ambitious-lzy, lvdongxu, wangganginjsu, Huygens, liyan.chen, wenjieli, jiangjianfei, sunyanan, guanghui.he}@sjtu.edu.cn

Abstract—With the widespread deployment of long-context large language models (LLMs), efficient and high-quality generation is becoming increasingly important. Modern LLMs employ batching and key-value (KV) cache to improve generation throughput and quality. However, as the context length and batch size rise drastically, the KV cache incurs extreme external memory access (EMA) issues. Recent LLM accelerators face substantial processing element (PE) under-utilization due to the low arithmetic intensity of attention with KV cache, while existing KV cache compression algorithms struggle with hardware inefficiency or significant accuracy degradation. To address these issues, an algorithm-architecture co-optimization, KVO-LLM, is proposed for long-context batched LLM generation. At the algorithm level, we propose a KV cache quantization-aware pruning method that first adopts salient-token-aware quantization and then prunes KV channels and tokens by attention guided pruning based on salient tokens identified during quantization. Achieving substantial savings on hardware overhead, our algorithm reduces the EMA of KV cache over 91% with significant accuracy advantages compared to previous KV cache compression algorithms. At the architecture level, we propose a multi-core jointly optimized accelerator that adopts operator fusion and cross-batch interleaving strategy, maximizing PE and DRAM bandwidth utilization. Compared to the state-of-the-art LLM accelerators, KVO-LLM improves generation throughput by up to 7.32 $\times$, and attains 5.52~8.38 $\times$ better energy efficiency.

Index Terms—Large language model, Long-context, Batching, KV cache, Algorithm-architecture co-design

I. INTRODUCTION

Long-context large language models (LLMs) show exceptional dexterity across a spectrum of natural language processing tasks with notable proficiency, especially in long-context generation [1]. With the increasing demand for long-context generation, the maximum context length has increased dramatically from 2K to millions of tokens. To improve the generation throughput, a prevalent strategy is to batch requests, exploiting data sharing [2], [3]. Moreover, LLMs broadly employ the pivotal *key-value (KV) cache* technique, which stores the intermediate attention keys and values to avoid heavy re-computation during long-context generation [4], [5].

However, with a drastic increase in batch size and context length, KV cache incurs extreme memory issues. As depicted in Fig. 1(a), the KV cache external memory access (EMA)

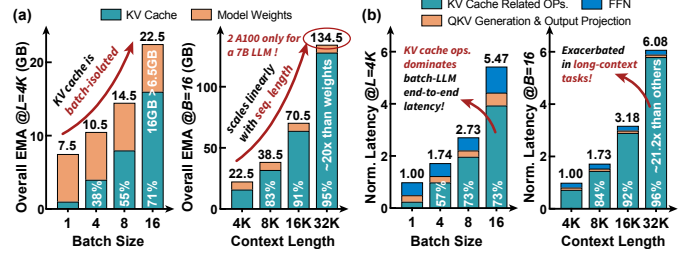


Fig. 1. Breakdown of EMA and latency for Llama-2-7B generating one token under different batch size B and context length L . Weights, activations, and KV cache are in INT8 format. The latency is measured on Tender [7] with a peak throughput of 2048GOPS@INT8 and a DRAM bandwidth of 256 GBps.

increases linearly with batch size. Taking Llama-2-7B [6] as an example, as the batch size rises, the EMA of KV cache exceeds that of the model weights under 4K context length. Furthermore, as shown in Fig. 1(b), the KV cache related operations predominantly influence the generation throughput under large batch size. With the increase of context length, the challenges become more pronounced. For example, the EMA of KV cache even increases to 128 GB when the context length reaches 32K, becoming more than 20 $\times$ the weight size. This extreme EMA accounts for 96% of the generation latency for Tender [7], forming a distinct bottleneck to throughput.

The above KV cache issues have not been adequately addressed in existing LLM accelerators [7]–[9]. Though they focus on lessening computational workload, there seems to be a common oversight regarding the low arithmetic intensity issues of KV cache. For example, Tender [7] and OPAL [8] quantize the LLMs into low precision to boost the processing element (PE) efficiency. MECLA [9] addresses the similar issue by compressing the weight matrices through fine-tuning. Nonetheless, in attention layers, their generation throughput is severely hindered by intensive EMA and low arithmetic intensity which causes serious PE under-utilization (e.g., only $\approx 25\%$ of Tender under 16-batched Llama-2-7B decoding [7]).

Many previous algorithms have been dedicated to compressing the KV cache through KV quantization (KV-Q) or KV eviction (KV-E). The state-of-the-art (SOTA) KV-Q works [10], [11] involve extra element-wise floating-point (FP) multiplication and accumulation (MAC) during matrix operations, which significantly degrades hardware efficiency. Meanwhile, other works of KV-E aim to retain a limited budget of important tokens, while evicting the trivial ones [12], [13]. However, discarded tokens might cause huge semantic information loss

[†]: Equal contribution.

This work was supported by the National Science Foundation of China under Grant 92464302. (*Corresponding author: Guanghui He).

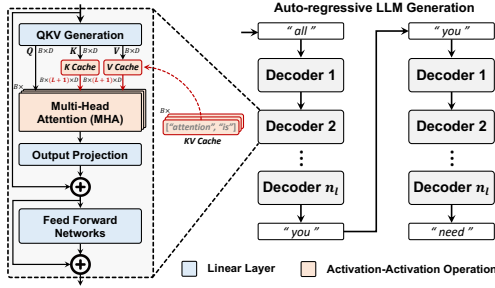


Fig. 2. Model architecture and auto-regressive inference of LLMs. As decoder layers focus on matrix computation, we exclude the layer normalization and position embedding in the diagram.

[14]. Despite its significant EMA savings, KV-E leads to noticeable accuracy loss on long-context generation [15].

In summary, KV cache emerges as a key barrier to high-throughput LLM inference while existing works leave a clear gap away from the memory-efficient design with high generation quality. Therefore, we consider three key requirements: ① *hardware efficiency* of the compression policy; ② *generation quality* under KV cache compression; and ③ *utilization maximization* of PE and DRAM bandwidth. To meet these requirements, we propose an algorithm-architecture co-optimization, **KVO-LLM**. The key contributions are as follows:

- 1) We propose an effective KV cache quantization-aware pruning algorithm, which adopts differential and salient-token-aware quantization (DSQ) firstly. Then based on the identified salient tokens, it employs historical-current attention guided pruning (HCAP) to prune KV channels and tokens. Our method not only achieves *substantial savings on hardware overhead* ①, but also delivers *high-quality generation comparable to that with full KV cache* ② under a high KV cache compression ratio (CR). This algorithm is open-sourced and available for public use.
- 2) We design a multi-core jointly optimized architecture with operator fusion and cross-batch interleaving strategy (OFBI) to schedule the compute flow wisely. Compared to naively scheduling our algorithmic flow, OFBI improves *PE utilization from 25% to 93% for multi-head attention and DRAM bandwidth usage from 25% to 67% for Query-Key-Value generation on average* ③, thus achieving up to $1.72\times$ throughput improvement.
- 3) We implement our design on TSMC 28nm technology and extensively evaluate it on various long-context generation tasks. KVO-LLM achieves comparable performance under a CR of over 91% as the model with full KV cache, while SOTA KV-Q and KV-E works still exhibit a notable accuracy loss even with a smaller CR. Compared with SOTA LLM accelerators, KVO-LLM improves generation throughput by up to $7.32\times$ and achieves $5.52\sim 8.38\times$ gain in energy efficiency.

II. BACKGROUND AND MOTIVATION

A. Large Language Models

The LLM inference flow is typically organized into two phases: the *prefilling* phase and the *decoding* phase. The former calculates and stores the KV cache of the input prompt,

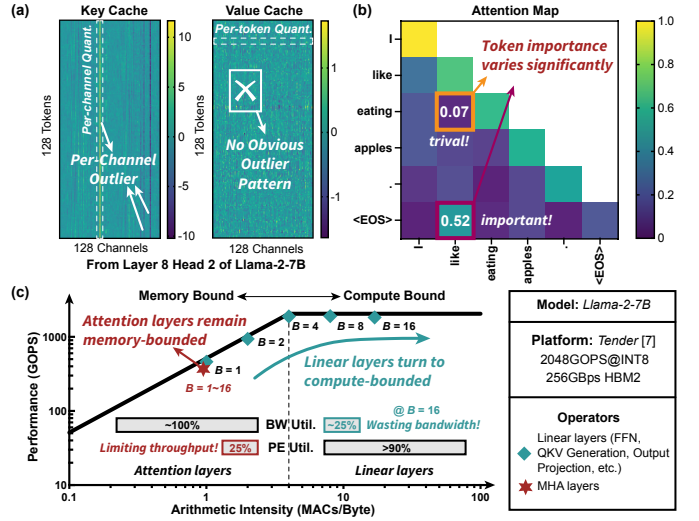


Fig. 3. (a) Magnitude of key and value cache from layer 8 head 2 of Llama-2-7B [6]. (b) The attention map of the input prompt “I like eating apples.” (c) Roofline model of the Tender [7] for the decoding phase of Llama-2-7B when L is 4096, where BW represents the DRAM bandwidth.

and generates the first token. The latter phase, as illustrated in Fig. 2, generates the output tokens one by one, updating the KV cache during each regression loop. Since the prefilling phase is widely researched [7]–[9] and accounts for a minor portion of latency under long-context generation relatively [1], this paper focuses primarily on the *decoding* phase.

Fig. 2 shows that LLM is mainly constructed by several stacked decoder layers, each comprising four major parts: (1) Query(Q)-Key(K)-Value(V) generation, (2) multi-head attention (MHA), (3) output projection, and (4) feed forward networks (FFNs). (1), (3), and (4) are composed of linear layers, while the MHA primarily involves multiplication between activations. Denote the batch size, current context length, and embedding dimension by B , L , and D , respectively. After QKV generation, the newly generated $\mathbf{K}_i, \mathbf{V}_i \in \mathbb{R}^{B \times D}$ need to be concatenated with the cached $\mathbf{K}_C, \mathbf{V}_C \in \mathbb{R}^{B \times L \times D}$:

$$\begin{aligned} \mathbf{K}, \mathbf{V} &= \text{Concat}(\mathbf{K}_C, \mathbf{K}_i), \text{Concat}(\mathbf{V}_C, \mathbf{V}_i), \\ \mathbf{K}_C, \mathbf{V}_C &= \mathbf{K}, \mathbf{V}, \end{aligned} \quad (1)$$

where $\mathbf{K}$ and $\mathbf{V}$ participate in the following MHA: attention score computation $\mathbf{S} = \text{Softmax}(\mathbf{Q}_i \mathbf{K}^T)$, and $\mathbf{x}_{out} = \mathbf{S} \mathbf{V}$.

B. Motivations

To compress KV cache, the most effective method is to reduce the total bytes it occupies, i.e., KV-Q. We profile the magnitude distribution of KV cache in Llama-2-7B [6] in Fig. 3(a). It could be observed that some $\mathbf{K}$ channels demonstrate large absolute magnitudes. Such features suggest a natural preference of per-channel $\mathbf{K}$ quantization [10]. However, this incurs an expensive overhead of FP MACs, as the $\mathbf{Q}_i \mathbf{K}^T$ computation of MHA is along the token dimension. Nevertheless, the per-token method, which is relatively hardware-friendly, introduces significant accuracy loss when quantizing vastly volatile channels [10]. Additionally, KV-E is an effective KV cache compression method by identifying and eliminating tokens with trivial context. However, as demonstrated in Fig.

3(b), the importance of tokens varies significantly across different context positions. Although research like H₂O [13] relies on historical accumulated attention scores to distinguish trivial tokens as precisely as possible, evicted tokens may become important in future generation, causing significant loss of semantic information [14]. Above all, the challenge for current KV cache compression lies in *ensuring both hardware efficiency and generation quality*.

Compared to the aforementioned algorithmic optimization, existing LLM accelerators often neglect the KV cache compute issues. We conduct a roofline analysis on Llama-2-7B in Fig. 3(c) based on Tender [7]. For the linear layers, batching improves the arithmetic intensity with weight sharing, whereas attention layers remain memory-bound consistently regardless of how batch size and context length increase. Divergent features determine that Tender suffers severe DRAM bandwidth or PE under-utilization in these two types of LLM layers respectively. As exhibited in Fig. 1(b), while KV cache scaling up with B and L , it exacerbates the aforementioned bottlenecks. Therefore, there is an urgent need for *the approach for PE efficiency and memory bandwidth maximization in batched LLM inference* to achieve high generation throughput.

III. PROPOSED KV CACHE COMPRESSION APPROACH

In this section, we propose an effective quantization-aware pruning algorithm combining differential and salient-token-aware quantization (DSQ) with historical-current attention guided pruning (HCAP). DSQ identifies salient tokens and achieves extremely low-bit KV cache quantization. Based on the identified salient tokens, HCAP prunes a great number of trivial channels and tokens to enhance the KV cache CR. In Sec. III-A, we first introduce the DSQ. Then the HCAP will be described in Sec. III-B. Finally, Algorithm 1 details each step of our KVO-LLM algorithm combining DSQ with HCAP.

A. Differential and Salient-Token-Aware Quantization

To minimize the FP MACs, it is essential to ensure that the quantization dimension align with the accumulation. Therefore, we should employ per-token $\mathbf{K}$ and per-channel $\mathbf{V}$ quantization. However, using it naively causes huge accuracy drop [10]. To overcome this dilemma, the DSQ is proposed.

1) *Differential Per-Token $\mathbf{K}$ Quantization*. As illustrated in Fig. 3(a), adjacent $\mathbf{K}$ exhibit high numerical similarity along the channel dimension. This property indicates that the differences between adjacent tokens are minimal. Leveraging this opportunity, we propose a differential quantization method that partitions all tokens into groups, each containing one base token ($\mathbf{K}_{base}$) and several Δ tokens ($\mathbf{K}_{\Delta}$). When quantizing each group, the $\mathbf{K}_{base}$ is directly quantized to INT8, while the $\mathbf{K}_{\Delta}$ are represented by the low-bit quantized differential values relative to the $\mathbf{K}_{base}$:

$$\text{Quant}(\mathbf{K}_{\Delta}) = \text{Quant}(\mathbf{K}_{\Delta} - \mathbf{K}_{base}, \text{INT2}) \quad (2)$$

A new challenge arises in selecting the $\mathbf{K}_{base}$ and implementing effective grouping strategies. According to [13], selecting tokens with the highest accumulated attention scores

Algorithm 1: Proposed KVO-LLM Algorithm

Input: Query $\mathbf{Q} \in \mathbb{R}^{L \times D}$, KV cache $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{L \times D}$, KV cache budget $k \in \mathbb{N}$, recent generation observation length R , window size n_{win} , pruning ratio P

Output: MHA results $\mathbf{x}_{out}$

```

1 Set  $\mathbf{S}_0 = 0$ ,  $\mathbf{S}_0 \in \mathbb{R}^L$ ; and  $i = 0$ ;
2 for  $i = 1 \sim L$  do
3    $\mathbf{Q}_i = \mathbf{Q}[i, :]$ 
4   // Pre-compute Stage;
5   // Sec. III-B: Current Attention Guided Pruning;
6   // Estimate current attention scores based on  $\mathbf{K}_{base}$ ;
7    $\mathbf{A}_{est} = \mathbf{Q}_i \mathbf{K}_{base}^T$ ;
8   Select  $k$  budget tokens  $\mathbf{K}_{bgt}$ ,  $\mathbf{V}_{bgt}$  based on  $\mathbf{A}_{est}$ ;
9   Select recent tokens  $\mathbf{K}_{rec}$ ,  $\mathbf{V}_{rec}$  based on  $R$ ;
10   $\mathbf{K}_s, \mathbf{V}_s = \text{Concat}(\mathbf{K}_{bgt}, \mathbf{K}_{rec}), \text{Concat}(\mathbf{V}_{bgt}, \mathbf{V}_{rec})$ ;
11  // Formal-compute Stage;
12   $\mathbf{S}_{cur} = \text{Softmax}(\mathbf{Q}_i \mathbf{K}_s^T)$ ;
13   $\mathbf{x}_{out} = \mathbf{S}_{cur} \mathbf{V}_s$ ;
14  // Accumulated with historical attention scores  $\mathbf{S}_{i-1}$ ;
15   $\mathbf{S}_i = \mathbf{S}_{i-1} + \mathbf{S}_{cur}$ ;
16  if  $(i \% R == 0)$  and  $(i \geq 2R)$  then
17    // Sec. III-A: Select base tokens;
18     $\mathbf{S}_{win} = \mathbf{S}_i[i - 2R : i - R]$ ;
19     $\mathbf{S}_{win} = \mathbf{S}_{win}.\text{reshape}(n_{win}, -1)$ ;
20     $slt_{idx} = \text{topk}(\mathbf{S}_{win})$ ; // Salient tokens index;
21     $\mathbf{K}_{cmps}, \mathbf{V}_{cmps} = \mathbf{K}, \mathbf{V}[i - 2R : i - R, :]$ ;
22    // Sec. III-A: DSQ;
23     $(\mathbf{K}_{cmps}, \mathbf{V}_{cmps}) = \text{Quant}(slt_{idx})$ ;
24    // Sec. III-B: Historical Attention Guided Pruning;
25     $(\mathbf{K}_{cmps}, \mathbf{V}_{cmps}) = \text{Prune}(slt_{idx}, P)$ ;
```

as base tokens like H₂O [13] is the most appropriate approach. However, this can lead to an uneven distribution in memory, resulting in irregular memory access. Therefore, we propose a window-based $\mathbf{K}_{base}$ selection strategy at line 18-20 in Algorithm 1. Every n_{win} tokens form a group, and within each group, the token with the highest accumulated score from the most recent R generations is identified as the salient token, with its corresponding attention key serving as $\mathbf{K}_{base}$.

2) *Salient-Token-Aware Per-Channel $\mathbf{V}$ Quantization*. As shown in Fig. 3(a), $\mathbf{V}$ does not exhibit an obvious outlier pattern like $\mathbf{K}$. As a result, for per-channel quantization, the precision of salient tokens is susceptible to the quantization process imposed on trivial tokens. Given that the result ($\mathbf{x}_{out}$) of $\mathbf{SV}$ primarily depends on the combination of a few important tokens, compared to $\mathbf{K}$ quantization, we do not employ a differential process on $\mathbf{V}$ to mitigate interactions between tokens. We allocate high bit-width (8-bit) for salient tokens, and assign low bit-width (2-bit) to trivial tokens.

B. Historical-Current Attention Guided Pruning

Further reduction in quantization bit-width can significantly impair accuracy. To enhance the CR, it is necessary to prune the trivial information within the KV cache. We rely on both historical accumulated attention scores and current attention scores, i.e., current $\text{Softmax}(\mathbf{Q}_i \mathbf{K}_{base}^T)$ results, to guide the pruning process and mitigate semantic information loss.

1) *Historical Attention Guided Pruning*. Evicting tokens thoroughly based on historical accumulated attention scores causes significant loss of semantic information. However,

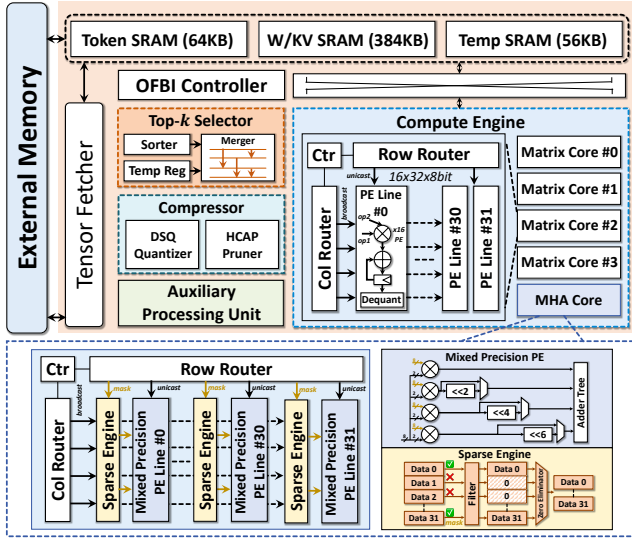


Fig. 4. Overview of the KVO-LLM accelerator.

there are many channels with small magnitudes of trivial tokens delivering negligible contribution to the final results. Therefore, we selectively prune these trivial channels at line 25 in Algorithm 1. Based on the absolute values, we adopt a structured pruning method that eliminates the lowest N values in each quantization group. When pruning ratio is $1 - P$, the compression ratio (CR) of one group is reduced to:

$$G_{CR}(P) = (8D + 2DP(n_{win} - 1))/16DL \quad (3)$$

2) *Current Attention Guided Pruning*. Large pruning ratio ($>75\%$) solely based on historical attention scores results in significant accuracy degradation. Fortunately, prior research demonstrates the inherent sparsity in attention scores [14]. Therefore, to mitigate accuracy loss while achieving a high CR, we propose to estimate the current attention scores and utilize them to select a small portion of tokens participating in current computation at line 7-8 in Algorithm 1. First, instead of multiplying Q_i with all K , we approximate attention scores by multiplying Q_i and K_{base} . The scores estimate the importance of each KV group. Then, given a KV cache budget k , only the top- (k/n_{win}) groups with the largest $Q_i K_{base}^T$ results participate in the following computations.

When $L\%R = 0$, the K cache EMA CR using our approach is reduced to

$$K_{CR} = 1 - \left(\underbrace{R/L}_{recent} + \underbrace{G/2L}_{base} + \underbrace{(k/n_{win})G_{CR}(P)}_{k \text{ budget}} \right), \quad (4)$$

where $G = (L - R)/n_{win}$, which is # of groups. For V_{CR} , it only includes the EMA of recent tokens and k budget tokens.

IV. PROPOSED GENERATIVE LLM ACCELERATOR

As depicted in Fig. 3(c), existing LLM-specific designs face severe PE or DRAM bandwidth under-utilization issues. We design a multi-core jointly optimized architecture which schedules the compute flow wisely leveraging operator fusion and cross-batch interleaving (OFBI) strategy. We first present the overview of our accelerator in Sec. IV-A. Then, we show

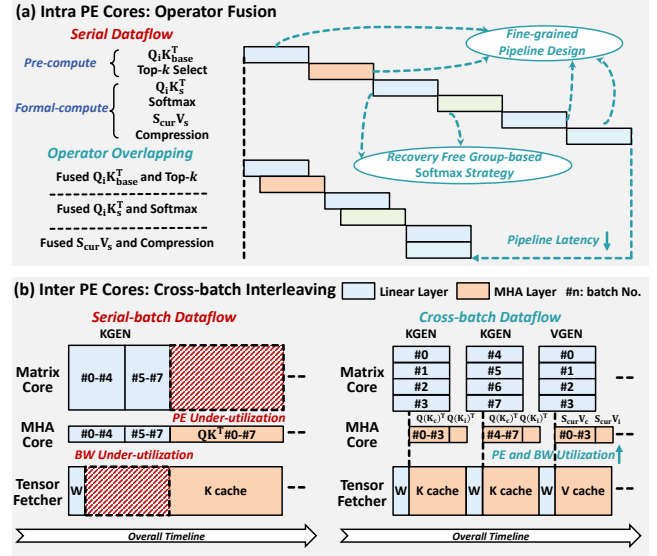


Fig. 5. Proposed operator fusion and cross-batch interleaving (OFBI) dataflow. (a) Intra PE cores operator fusion to reduce pipeline latency. (b) Inter PE cores cross-batch interleaving to improve PE and DRAM bandwidth utilization.

how operators are fused within one core in Sec. IV-B. The cross-batch interleaving strategy is described in Sec. IV-C.

A. Overview

Fig. 4 depicts the overall architecture of KVO-LLM. It primarily consists of 504KB on-chip buffer, an auxiliary processing unit, a compute engine, and specialized functional units. The on-chip buffer is divided into three main parts for input feature maps, weights/KV cache, and temporary results respectively. The auxiliary processing unit is mainly responsible for Softmax and other non-linear functions. The compute engine is composed of five PE cores focused on matrix multiplication, including one MHA core dedicated to multi-head attention. The specialized functional units include a top- k selector and a compressor to support our KV cache quantization-aware pruning algorithm.

B. Intra PE Cores: PE Design and Operator Fusion

As shown in Fig. 4, each PE core consists of 32 PE lines, and each PE line is composed of 16 PEs. Each PE core performs $(1 \times 16) \times (16 \times 32)$ matrix-vector multiplication per cycle, and it follows an output stationary dataflow, where partial sums are accumulated in each PE line. The MHA core is composed of precision-configurable PEs and sparse engines based on zero elimination to support mixed-precision quantization and pruning respectively.

When performing MHA in a single batch, our algorithm necessitates data recovery of $Q_i K_{base}^T$ because of DSQ, top- k selection because of HCAP, and compression of new KV tokens. Naively performing these operations in a serial manner causes long pipeline latency, which could be seen in Fig. 5(a). We propose a fine-grained fusion dataflow to improve PE cores utilization significantly. In the pre-compute stage, once a valid $Q_i K_{base}^T$ result is finished, the top- k selector compares it with previous results. In the formal-compute stage, the Softmax

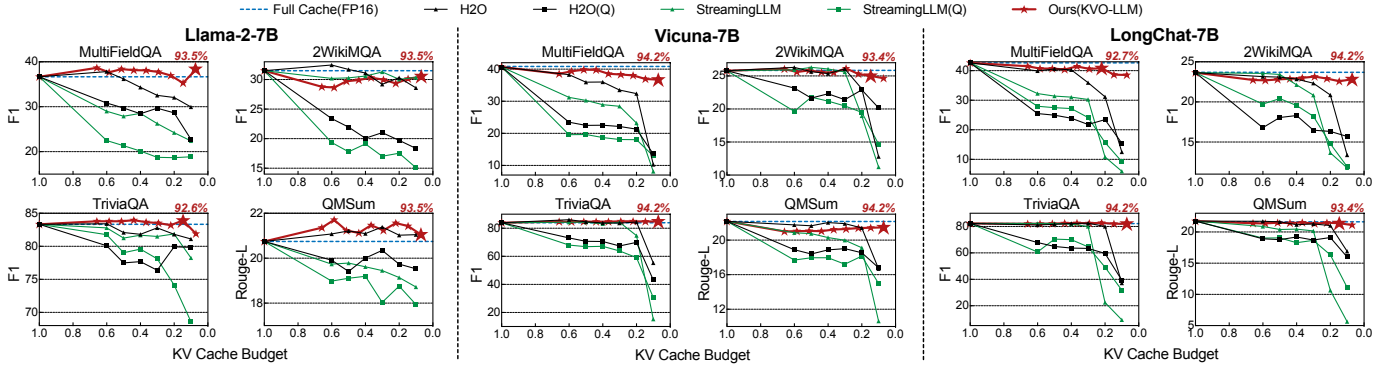


Fig. 6. Comparison results between the baseline model with full cache, our KVO-LLM algorithm, and the existing KV-E methods [12], [13] with or without quantization. We choose 4 representative datasets for distinct tasks: single-document QA, multi-document QA, synthetic tasks, and summarization, respectively, from the extensively utilized LongBench [15]. For each diagram, we highlight one point with significant CR and high accuracy compared to SOTA works, and we present the actual CR. Our method could compress the KV cache EMA over 91% while demonstrating negligible accuracy loss.

can overlap with $\mathbf{Q}_i \mathbf{K}_s^T$ by fusing data recovery process with the Softmax operation and utilizing a group-based Softmax strategy. More specifically, we implement Softmax for each group of $\mathbf{Q}_i \mathbf{K}_s^T$, i.e., n_{win} results. By designating $\mathbf{Q}_i \mathbf{K}_{base}^T$ as the Softmax maximum, we eliminate the necessity for completing data recovery. After finishing $\mathbf{Q}_i \mathbf{K}_s^T$, we update group-based Softmax results accordingly. Then the compute engine begins $\mathbf{S}_{cur} \mathbf{V}_s$. Compression on new KV tokens can be operated simultaneously on the specially designed compressor.

C. Inter PE Cores: Cross-Batch Interleaving

As shown in Fig. 3 (c), in the linear layers, batching enhances arithmetic intensity, while attention layers consistently exhibit memory bound, irrespective of batch size and context length. Fig. 5(b) presents an unoptimized execution timeline for the multi-batch computational process, which executes the QKV generation and MHA serially. This execution flow leads to severe under-utilization of both PE and DRAM bandwidth.

Therefore, we introduce a cross-batch interleaving approach. Fig. 5(b) illustrates an example of this technique, which is accomplished by partitioning the compute cores into separate matrix and MHA cores to accommodate distinct workloads. Since there is no dependency between KV generation and MHA with cached $\mathbf{K}_C, \mathbf{V}_C$, MHA is divided into two computational segments: MHA with cached $\mathbf{K}_C, \mathbf{V}_C$ and MHA with new $\mathbf{K}_i, \mathbf{V}_i$. We dispatch them with other batches to optimize resource utilization. To be specific, because of the compute-bounded property in batched KV generation, there is a long period of DRAM idleness. Instead of scheduling all compute cores to operate on KV generation, we schedule one MHA core for MHA with cached $\mathbf{K}_C, \mathbf{V}_C$, fully utilizing the wasted DRAM bandwidth. Cross-batch interleaving facilitates the concurrent execution of compute-intensive linear layers alongside memory-bound attention layers, and thus effectively improves PE and DRAM bandwidth utilization.

V. EVALUATION

A. Evaluation Methodology

We evaluate KVO-LLM on four datasets in the widely-used LongBench [15] to demonstrate our superiority over diverse

TABLE I
COMPARISONS WITH QUANT-ONLY STRATEGIES

LLM	KV Quant	Evaluation Scores			KV CR (%)
		MultiFieldQA	2WikiMQA	QMSum	
Llama-2-7B	FP16	36.70	31.51	20.75	0
	NINT2 [†]	28.49	20.09	19.70	87.50
	KIVI*	35.07	29.04	20.70	85.90
	Ours**	37.74	29.99	21.24	90.89
LongChat-7B	FP16	42.66	23.66	21.75	0
	NINT2 [†]	32.94	20.65	19.19	87.50
	KIVI*	38.79	22.74	21.64	86.10
	Ours**	40.63	23.17	21.36	91.93
Vicuna-7B	FP16	40.68	25.79	22.19	0
	NINT2 [†]	23.36	19.96	19.00	87.50
	KIVI*	34.65	25.36	21.42	86.10
	Ours**	38.34	26.16	21.78	91.93

[†] NINT2: Naive per-token INT2 quantization for KV cache following the same group granularity as KVO-LLM.

* KIVI: Except for setting the residual length to 64, all other configurations remain the same as in the original paper.

** Our results are tested under $k = 896, R = 64, P = 50\%$.

tasks, e.g., document QA, summarization, and synthetic tasks. LongBench provides a holistic evaluation of long-context generation abilities of LLMs across various lengths, distributions, and long dependency patterns [15]. Detailed configurations and implementation are derived from the official repository of H2O [13] and KIVI [10]. Meanwhile, three representative long-context LLMs are chosen for our evaluation: Llama-2-7b-chat-4k (Llama-2-7B) [6], LongChat-v1.5-7b-32k (LongChat-7B) [16], Vicuna-7b-v1.5-16k (Vicuna-7B) [17]. When evaluating accuracy, we use the PyTorch [18] framework and the Hugging Face library [19].

For hardware implementation, we implement the compute and control logic of KVO-LLM with SystemVerilog and estimate the area and energy consumption of on-chip memory with CACTI [20]. The prototype is synthesized with Synopsys Design Compiler under TSMC 28nm CMOS technology. Logic power consumption is estimated using Synopsys PrimeTimePX by annotating switching activity from the datasets above. For off-chip DRAM, we use an HBM2 with 256GB/s bandwidth, which consumes 1.2pJ/b for data access [21].

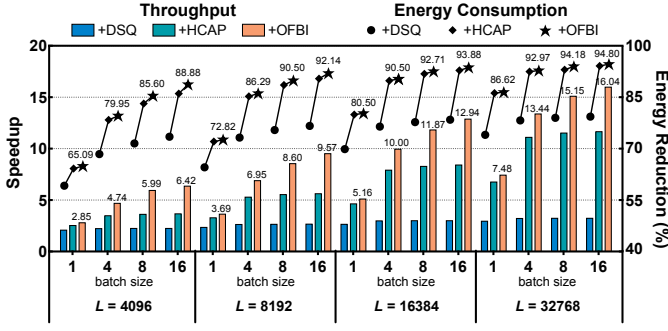


Fig. 7. Normalized throughput and energy consumption improvements through DSQ, HCAP and OFBI. Note that “+” means this method is appended on the previous approach. Baseline has the same hardware architecture with these functions deactivated.

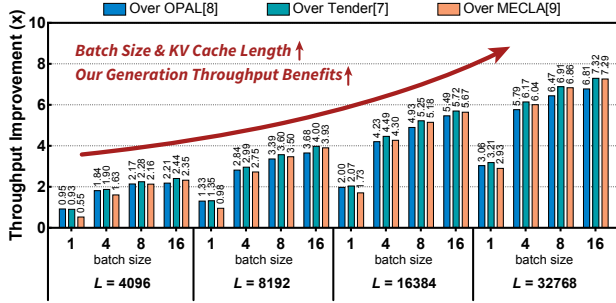


Fig. 8. Normalized throughput improvements of KVO-LLM with SOTA LLM accelerators [7]–[9]. We ensure the peak throughput is consistent across all works to make a fair comparison.

B. Evaluation on Proposed Algorithm

We first evaluate our method with SOTA KV-Q works, such as KIVI. As shown in Table I, NINT2 suffers from significant accuracy loss due to the outlier channels in **K**. Although KIVI mitigates the quantization error by adjusting the quantization dimensions, it introduces inefficient FP MACs. Compared to KIVI with up to a 14.82% accuracy drop and an 86.10% KV cache CR, KVO-LLM only has up to a 5.76% accuracy drop despite achieving up to 91.93% CR. KVO-LLM outperforms existing methods on both accuracy and KV cache CR.

We further compare our algorithm with SOTA KV-E works across different KV cache budget in Fig. 6. To clearly demonstrate our strengths, the KV cache budget shown in the figure is calculated as k/L . However, with the inclusion of DSQ and HCAP, the CR using our method significantly surpasses $1 - k/L$, as calculated in (4). We can observe that: (1) KVO-LLM with a CR above 91% can achieve comparable accuracy to the model with full KV cache (FP16). (2) KVO-LLM shows increasingly significant accuracy advantages under high CR. (3) Combining DSQ and HCAP, KVO-LLM demonstrates consistent and significant accuracy improvements over KV-E works with quantization.

C. Evaluation on Proposed Hardware Accelerator

Fig. 7 presents the throughput and energy consumption evaluations over three key techniques: DSQ, HCAP, and OFBI. We set $k = L/8$ for the following evaluations.

When compared to the baseline, DSQ and HCAP yield over $3.72\times$ enhancement in throughput and over 86.3% reduction in

TABLE II
COMPARISON WITH OTHER LLM ACCELERATORS

	OPAL [8]	Tender [7]	MECLA [9]	Ours
Technology(nm)	65	28	28	28
Frequency(MHz)	500	1000	1000	500
Area(mm ²)	0.69 [‡]	3.98	22.02	4.83
Memory(KB)	514	608	1280	504
Throughput(GOPS)*	71	1328	1460	3287
Power(mW)	700	1600	1976	717
Energy Efficiency [†] (GOPS/W)	546.60	830.00	738.87	4584.38

* Evaluation on Llama-2-7B under $L = 4096$, $B = 16$. [‡] Without SRAM.

[†] Technology normalized to 28nm as follows: $s = \text{Technology}/28\text{nm}$, $f \sim s$, and $P \sim (1/s)$, where f and P is denoted as frequency and power, respectively.

power consumption under $B = 16$, $L = 4096$. The introduced OFBI strategy further improves throughput by up to $1.72\times$ and decreases power consumption by up to $1.24\times$. Additionally, as illustrated in Fig. 7, the performance improvement is typically greater with larger batch size and context length, as these conditions elevate the latency and EMA contribution from the MHA stage within the processing flow.

D. Comparison with SOTA LLM Accelerators

We conduct a comparison of KVO-LLM with leading LLM accelerators [7]–[9], and ensure the peak throughput is consistent across all works. Compared with them, KVO-LLM effectively reduces the EMA of KV cache with a CR $>91\%$ by employing DSQ and HCAP. Moreover, utilizing the OFBI, KVO-LLM significantly improves PE utilization from 25% to 93% for MHA and DRAM bandwidth usage from 25% to 67% for QKV generation. As a result, KVO-LLM improves throughput by up to $7.32\times$ in Fig. 8.

Table II presents the detailed implementation results. Utilizing TSMC 28nm technology, the KVO-LLM prototype demonstrates an effective performance of 4584.38 GOPS/W at 500 MHz, with a power consumption of 717 mW. The accelerator occupies an area of 4.83 mm^2 . We also reconstruct the method of [7]–[9] for the LLM generation phase to evaluate their energy efficiency. Leveraging DSQ, HCAP and OFBI, KVO-LLM shows a $5.52\sim 8.38\times$ increase in energy efficiency compared to these accelerators.

VI. CONCLUSION

This paper introduces an algorithm-architecture co-optimization, KVO-LLM. The key insight is to alleviate the significant KV cache EMA and improve the throughput for long-context batched LLM generation. Through quantization-aware pruning, KVO-LLM not only achieves substantial savings on hardware overhead, but also compresses KV cache EMA by over 91% with high generation quality. Besides, KVO-LLM hardware accelerator integrates the OFBI strategy to maximize PE and DRAM bandwidth utilization. Comprehensive evaluations demonstrate KVO-LLM surpasses SOTA works in various metrics such as accuracy, throughput, and energy efficiency across diverse tasks and model types.

REFERENCES

- [1] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, S. Li, Y. Lou, L. Wang, Z. Yuan, X. Li, S. Yan, G. Dai, X. Zhang, Y. Dong, and Y. Wang, "A Survey on Efficient Inference for Large Language Models," *CoRR*, vol. abs/2404.14294, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2404.14294>
- [2] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, R. Gupta, N. B. Abu-Ghazaleh, M. Musuvathi, and D. Tsafir, Eds. ACM, 2024, pp. 722–737. [Online]. Available: <https://doi.org/10.1145/3620665.3651380>
- [3] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "AttAcc! Unleashing the Power of PIM for Batched Transformer-based Generative Model Inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, R. Gupta, N. B. Abu-Ghazaleh, M. Musuvathi, and D. Tsafir, Eds. ACM, 2024, pp. 103–119. [Online]. Available: <https://doi.org/10.1145/3620665.3640422>
- [4] W. Lee, J. Lee, J. Seo, and J. Sim, "InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management," in *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, A. Gavrilovska and D. B. Terry, Eds. USENIX Association, 2024, pp. 155–172. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/lee>
- [5] Y. Zhao, D. Wu, and J. Wang, "ALISA: Accelerating Large Language Model Inference via Sparsity-Aware KV Caching," in *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*. IEEE, 2024, pp. 1005–1017. [Online]. Available: <https://doi.org/10.1109/ISCA59077.2024.00077>
- [6] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, "Llama 2: Open Foundation and Fine-Tuned Chat Models," 2023. [Online]. Available: <https://arxiv.org/abs/2307.09288>
- [7] J. Lee, W. Lee, and J. Sim, "Tender: Accelerating Large Language Models via Tensor Decomposition and Runtime Requantization," in *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*. IEEE, 2024, pp. 1048–1062. [Online]. Available: <https://doi.org/10.1109/ISCA59077.2024.00080>
- [8] J. Koo, D. Park, S. Jung, and J. Kung, "OPAL: Outlier-Preserved Microscaling Quantization Accelerator for Generative Large Language Models," in *Proceedings of the 61st ACM/IEEE Design Automation Conference, ser. DAC '24*. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3649329.3657323>
- [9] Y. Qin, Y. Wang, Z. Zhao, X. Yang, Y. Zhou, S. Wei, Y. Hu, and S. Yin, "MECLA: Memory-Compute-Efficient LLM Accelerator with Scaling Sub-matrix Partition," in *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*. IEEE, 2024, pp. 1032–1047. [Online]. Available: <https://doi.org/10.1109/ISCA59077.2024.00079>
- [10] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache," in *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=L057s2Rq8O>
- [11] Z. Zhang, S. Liu, R. Chen, B. Kailkhura, B. Chen, and A. Wang, "Q-Hitter: A Better Token Oracle for Efficient LLM Inference via Sparse-Quantized KV Cache," in *Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024*, P. B. Gibbons, G. Pekhimenko, and C. D. Sa, Eds. mlsys.org, 2024. [Online]. Available: https://proceedings.mlsys.org/paper/_files/paper/2024/hash/bbb7506579431a85861a05fff048d3e1-Abstract-Conference.html
- [12] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient Streaming Language Models with Attention Sinks," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=NG7sS51zVF>
- [13] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. W. Barrett, Z. Wang, and B. Chen, "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., 2023. [Online]. Available: http://papers.nips.cc/paper/_files/paper/2023/hash/6ceef7b15572587b78ecfceb2827f8-Abstract-Conference.html
- [14] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, "QUEST: Query-Aware Sparsity for Efficient Long-Context LLM Inference," in *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=KzACYw0MTV>
- [15] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, "LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, L. Ku, A. Martins, and V. Srikumar, Eds. Association for Computational Linguistics, 2024, pp. 3119–3137. [Online]. Available: <https://doi.org/10.18653/v1/2024.acl-long.172>
- [16] D. Li, X. Shao, A. Xie, Y. Sheng, L. Zheng, J. E. Gonzalez, I. Stoica, X. Ma, and H. Zhang, "How Long Can Open-Source LLMs Truly Promise on Context Length?" June 2023. [Online]. Available: <https://lmsys.org/blog/2023-06-29-longchat/>
- [17] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez, I. Stoica, and E. P. Xing, "Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90% Chatgpt Quality," March 2023. [Online]. Available: <https://lmsys.org/blog/2023-03-30-vicuna/>
- [18] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, "Automatic differentiation in PyTorch," 2017.
- [19] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and b. . P. o. t. . C. o. E. M. i. N. L. P. S. D. Alexander M. Rush", "Transformers: State-of-the-Art Natural Language Processing." Online: Association for Computational Linguistics, oct 2020, pp. 38–45. [Online]. Available: <https://www.aclweb.org/anthology/2020.emnlp-demos.6>
- [20] N. Muralimanohar, R. Balasubramanian, and N. P. Jouppi, "Optimizing NUCA Organizations and Wiring Alternatives for Large Caches with CACTI 6.0," in *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-40 2007), 1-5 December 2007, Chicago, Illinois, USA*. IEEE Computer Society, 2007, pp. 3–14. [Online]. Available: <https://doi.org/10.1109/MICRO.2007.33>
- [21] M. O'Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, "Fine-Grained DRAM: Energy-Efficient DRAM for Extreme Bandwidth Systems," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture, ser. MICRO-50 '17*, New York, NY, USA, 2017, p. 41–54.